Artificial Intelligence for Technical Leaders

AI for Tech Leaders
From Awareness to Execution

Program Objectives

By the end of this program, leaders will be able to:

- Understand what AI, ML, GenAI, LLMs, and Agents are, and what they **cannot** do
- Know where AI fits across the **SDLC**: requirements → dev → test → ops → support
- Evaluate AI investments: **ROI, FinOps, pricing**, own LLM vs. integrate
- Gain hands-on exposure to **building and using AI tools**
- Establish **governance, security, and adoption** principles
- Create real **internal use-cases** for their teams

AI 101 for Engineering Leaders
Building a shared mental model

What Is the Core Idea?

At the heart of all AI is one desire:

Find a function f that maps inputs X to outputs Y

- $f(x) = 2x + 3$, simple, hand-coded rule
- f(email text) = spam or not spam, needs learning from data
- f(photo) = face identity, too complex for rules, needs Deep Learning
- f("write me a summary") = paragraph, needs a Large Language Model

Intuition: Traditional programming is *you telling the machine the rules*. Machine Learning is *the machine figuring out the rules from examples*.

What is AI?, A Working Definition

Artificial Intelligence (AI):

- Machines performing tasks that *normally require human intelligence*
- Thinking, reasoning, recognizing patterns, learning, generating content

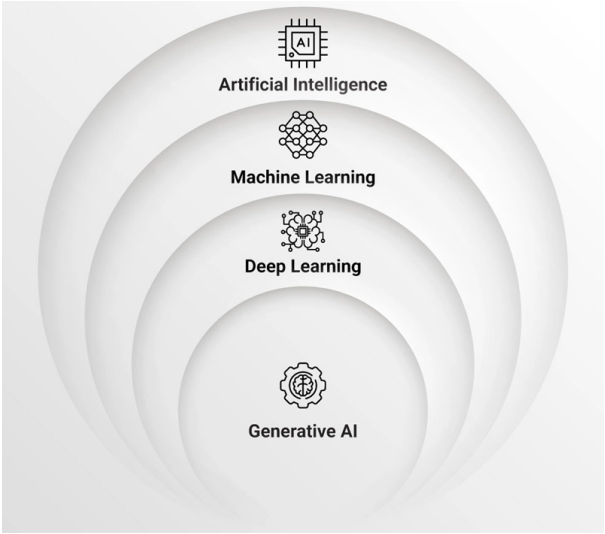
My definition: "If a computer program starts doing intelligent tasks, perceiving, reasoning, deciding, creating, that's AI."

Key Differentiators from traditional software:

- Traditional software: *explicit rules* → output
- AI/ML: *data + algorithm* → the rules emerge automatically

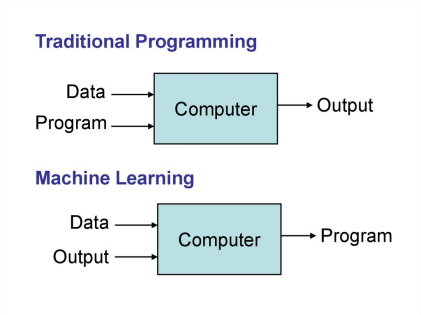
The Modern AI Hierarchy

- **AI (broadest):** Any machine mimicking human intelligence
- **ML (subset of AI):** Learns patterns from data, no explicit programming
- **Deep Learning (subset of ML):** Neural networks with many layers; powers vision, speech
- **Generative AI (subset of DL):** Creates *new* content, text, code, images, audio
- **LLMs (a type of GenAI):** Foundation models trained on massive text; e.g., GPT-4, Claude, Llama
- **Agents (built on LLMs):** LLMs given tools and memory to take autonomous actions



(Ref: Synoptek, NVIDIA)

Traditional Programming vs. Machine Learning



Key insight for engineering leaders: ML inverts the loop, you feed data and desired outputs, the algorithm finds the program.

Types of Machine Learning

- **Supervised Learning:** Learn from *labeled* examples (input + correct answer)
  - Example: Email → spam / not-spam; image → tumor / benign
- **Unsupervised Learning:** Find hidden structure in *unlabeled* data
  - Example: Cluster customers into segments without pre-defined groups
- **Reinforcement Learning:** Agent learns by *reward and penalty* from actions
  - Example: AlphaGo mastered chess by playing millions of games against itself
- **Self-supervised / Foundation Models:** Learn from raw internet text by predicting missing words, the basis of all modern LLMs

What AI Is Good At, and What It Is Not

AI excels at:

- Pattern recognition at scale (images, text, logs)
- Text generation, summarization, translation
- Code generation and review
- Anomaly detection in large data
- Repetitive classification tasks
- Personalization and ranking

AI struggles with:

- True reasoning and causality
- Precise arithmetic (hallucinations!)
- Real-time information (unless RAG/tools added)
- Emotional nuance and ethics
- Tasks with very little training data
- Explainability and auditability

Leader Rule: Clear deterministic rules? Use traditional code. Complex patterns in large data? Use AI.

Myths vs. Reality

- **Myth:** AI understands language like a human does. **Reality:** LLMs predict the *statistically most likely next token*, they do not "understand."
- **Myth:** AI is always accurate. **Reality:** Models hallucinate, they generate plausible-sounding but wrong answers with confidence.
- **Myth:** More AI = less engineers. **Reality:** Engineers who use AI well 2×–10× more productive; demand for skilled engineers rises.
- **Myth:** You need to train your own model. **Reality:** For 95% of enterprise use cases, you *integrate* existing LLMs via API + RAG.
- **Myth:** Enterprise AI and consumer AI are the same. **Reality:** Enterprise AI needs security, data isolation, compliance, cost controls.

Enterprise vs. Consumer AI

Aspect	Consumer	Enterprise
Example	ChatGPT free	Azure OpenAI, Bedrock
Data Privacy	Shared/Public	Isolated, contractually protected
Customization	Prompt only	Fine-tuning, RAG, system prompts
Cost	Free/flat	Token-based; FinOps needed
Compliance	None	SOC2, GDPR, HIPAA
Integration	None	SDLC, ticketing, monitoring
Reliability	Best-effort	SLA guaranteed

AI in Engineering Orgs, Where It Is Already

- **Code completion:** GitHub Copilot, Cursor, 30–40% developer time saved (McKinsey 2024)
- **PR review:** Automated review bots flagging bugs before human review
- **Test generation:** AI writes unit tests from function signatures
- **Incident response:** AI suggests RCAs from log patterns
- **Documentation:** Auto-generates API docs, release notes
- **Support triage:** LLM classifies and routes support tickets

The question is no longer "should we?", it's "how fast?"

Evolution Timeline

- 1950: Turing Test proposed
- 1980s: Backpropagation re-discovered; neural networks re-emerge
- 1997: Deep Blue beats Kasparov (Chess)
- 2012: AlexNet wins ImageNet, Deep Learning breakthrough
- 2016: AlphaGo beats world champion Lee Sedol
- 2017: *Attention Is All You Need*, Transformer architecture born
- 2018–2020: BERT, GPT-1, GPT-2, GPT-3
- 2022: ChatGPT launches, AI goes mainstream overnight
- 2023: GPT-4, Claude, Gemini, multimodal LLMs
- 2024: AI Agents, RAG mainstream; AlphaFold wins Nobel Prize
- 2025: Agentic AI in every SDLC phase; Physical AI emerging

LLMs, SLLMs & Agents

How they work, which to use, and when

What Is a Large Language Model (LLM)?

How it works (college-level intuition):

- An LLM is trained to predict: *"Given these words, what word comes next?"*
- Mathematically: given token sequence $w_1, w_2, \ldots, w_n$, compute $P(w_{n+1} \mid w_1 \ldots w_n)$
- **What this means in plain English:** The model reads billions of web pages and books, and builds an internal "map" of which words and ideas go together. When you ask it a question, it generates the most likely continuation of your text.
- During training on trillions of tokens, the model learns grammar, facts, reasoning patterns, and code, all from next-word prediction alone.

Size matters: "Large" = billions of parameters (weights). GPT-4 ≈ 1 trillion parameters.

How Transformers Work, The Intuition

The key innovation: Attention Mechanism

- Before Transformers: RNNs read text word-by-word, forgetting early context
- Transformers read the *entire sequence at once* and compute which words to *pay attention to*
- **Formula:** $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$
- **What this means:** For each word, calculate a "relevance score" with every other word, then mix their meanings proportionally. The word "bank" in "river bank" pays more attention to "river" than "money."

Result: Transformers handle long documents, code, and multi-turn conversations much better than older models.

The Modern LLM Landscape

Proprietary (API-access, closed weights):

- **OpenAI:** GPT-4o, o1, o3, best general reasoning
- **Anthropic:** Claude 3.5 / 3.7 Sonnet, best for coding, long context
- **Google:** Gemini 2.0 Flash, native multimodal, fast, cheap
- **Microsoft:** Azure OpenAI, enterprise-grade, compliance

Open Weights (self-hostable):

- **Meta:** Llama 3.3 70B, strong open alternative
- **Mistral:** Mistral Large 2, Codestral, great for code
- **DeepSeek:** DeepSeek-V3, strong reasoning, low cost

Key insight for leaders: Models are commoditizing. Your competitive advantage is in *how you integrate them*, not which model you pick.

Small LLMs (SLLMs), When Smaller Is Better

What is an SLLM? A compact model (1B–13B parameters) designed for specific tasks or on-device deployment.

- **Examples:** Phi-3 Mini, Gemma 2B, Llama 3.2 3B, Mistral 7B
- **When to use SLLM over LLM:**
  - Latency is critical (real-time, embedded systems)
  - Data must not leave your network (on-premise requirement)
  - Task is narrow and well-defined (e.g., classify support ticket type)
  - Cost is a primary constraint (1000× cheaper per token)
- **When to use a large LLM:**
  - Complex multi-step reasoning required
  - Long context window needed (legal docs, large codebases)
  - You need the best possible quality without cost sensitivity

Rule: Start with the smallest model that meets your quality bar.

What Is an AI Agent?

Chatbot vs. Agent, the critical distinction:

- **Chatbot:** One-shot Q&A. User asks → LLM replies. No memory, no tools, no action.
- **Agent:** LLM given *tools* (web search, code execution, APIs, databases) and a *goal*. It **plans**, executes steps, observes results, and adapts, autonomously.

Agent Loop (ReAct pattern):

Observe → Think → Act → Observe → . . . → Goal Achieved

Engineering examples:

- "Analyze this failing build, find root cause in logs, create a JIRA ticket.", 3 tool calls, no human in loop
- "Review this PR for security issues, check OWASP top 10, comment inline.", Agent with code analysis tool

Agentic Workflows Across the SDLC

- **Requirements & Design:** Requirement refinement agent, takes user stories, asks clarifying questions, generates acceptance criteria, flags ambiguities
- **Coding:** Coding agent, given a ticket, writes code, runs tests locally, raises PR (e.g., Devin, Claude Code)
- **Test Generation:** Agent generates unit tests, integration tests, edge cases from function signatures or API specs
- **Code Review:** PR review agent, checks style, logic bugs, security (OWASP), performance, adds inline comments
- **Release Notes:** Agent reads merged PRs + commit messages → drafts structured release notes
- **Ops & Support:** RCA agent, reads logs, traces, metrics → identifies root cause → suggests fix

Live Demo Concept: Requirement → Code → Test → Docs

The full SDLC in one agentic pipeline:

1. **Input:** "Build a REST endpoint that returns user profile by ID"
2. **Agent Step 1, Design:** LLM clarifies, generates OpenAPI spec
3. **Agent Step 2, Code:** LLM writes `FastAPI` endpoint with error handling
4. **Agent Step 3, Tests:** LLM writes `pytest` unit + integration tests
5. **Agent Step 4, Docs:** LLM writes README section + Swagger annotations
6. **Agent Step 5, Review:** LLM does security review (no SQL injection, input validation)

Time: ≈ 3–5 minutes. Human oversight: review and approve. **Key lesson:** The engineer's job shifts from *writing* to *reviewing and directing*.

MCP, Model Context Protocol

What is MCP? An open standard (Anthropic, 2024) that defines how LLMs connect to external tools and data sources.

- **Problem before MCP:** Every AI tool had custom integrations. Fragile, hard to maintain.
- **MCP solution:** Standardized protocol, LLM ↔ Tool (like USB for AI)
- **MCP architecture:**
  - **MCP Host:** The AI app (e.g., Claude, Cursor)
  - **MCP Server:** A connector to a specific tool (GitHub, Jira, Postgres, filesystem)
  - **MCP Client:** Bridge inside the host that speaks the protocol
- **Engineering relevance:** Build one MCP server for your internal tool, it works with any MCP-compatible AI agent

Analogy: Before: proprietary chargers for every phone. After: USB-C for all.

Enterprise AI Tooling & Policy

Setting guardrails before you build

Enterprise AI Tooling Stack

- **Model Access Layer:** Azure OpenAI / AWS Bedrock / Google Vertex AI, enterprise-grade APIs with compliance, rate limits, VPC
- **Orchestration:** LangChain, LlamaIndex, LangGraph, chain LLM calls, manage memory, build agent workflows
- **Vector Databases:** Pinecone, Weaviate, pgvector, Chroma, store embeddings for semantic search / RAG
- **Observability:** LangSmith, Langfuse, Phoenix, trace every LLM call, measure quality, debug failures
- **Deployment:** Docker + Kubernetes for containerized AI services; model serving via vLLM / Ollama for open models
- **IDE Integration:** GitHub Copilot, Cursor, Codeium, developer productivity layer

AI Usage Policy, What Must Be Defined

Your org needs clear answers to:

- **Data Classification:** What data can go to external LLM APIs? (PII, IP, secrets, never!)
- **Approved Tools:** Which AI tools are pre-approved? Which require a security review?
- **Output Ownership:** Who owns AI-generated code? How is it reviewed before merge?
- **Audit Trail:** Are LLM interactions logged? Can they be replayed for compliance?
- **Model Selection:** Is there a preferred model per use case? Who approves new models?
- **Access Control:** Who can deploy AI services? Who approves production use of agents?

Suggested starting point: An internal "AI Use Policy" document, reviewed by Security + Legal + Engineering leads. Treat AI like you treat external SaaS.

Key Security Risks in Enterprise AI

- **Data Leakage:** Developer pastes internal credentials or PII into ChatGPT, it may be used for training
  - *Mitigation:* Use enterprise API (data not used for training), DLP tools on clipboard
- **Prompt Injection:** Malicious input tricks the AI agent into performing unauthorized actions
  - *Mitigation:* Input sanitization, tool-call allowlists, human approval for destructive actions
- **Hallucinations in Production:** Model confidently returns wrong API schema, wrong config
  - *Mitigation:* Retrieval-Augmented Generation (RAG) grounding, human review for critical decisions
- **Supply Chain Risk:** Malicious or poisoned open-source model weights
  - *Mitigation:* Only use models from verified registries (Hugging Face verified, official providers)

RAG Architecture & Vector Databases

Making LLMs know about YOUR data

Why RAG? The Core Problem

The problem with raw LLMs:

- LLMs are trained on public internet data up to a cutoff date
- They know *nothing* about your internal runbooks, your codebase, your product docs
- Fine-tuning is expensive ($10k–$100k+) and needs to be re-done for every update

RAG = Retrieval-Augmented Generation

- Idea: At query time, *retrieve* relevant documents from your knowledge base and *inject* them into the LLM prompt as context
- The LLM now has your private data, without fine-tuning, without data leakage
- Result: LLM answers based on *your* documents, not just its training data

Analogy: Imagine giving an expert consultant your internal wiki right before the meeting, instead of hoping they already know it.

RAG Architecture, End to End

Indexing Phase (one-time):

1. Load documents (PDFs, Confluence, code, Slack threads)
2. Split into chunks (e.g., 500 tokens each with 50-token overlap)
3. Convert each chunk to a vector embedding: $\vec{v} = \text{Embed}(\text{chunk})$ *What is an embedding? A list of 1500 numbers that captures meaning. Similar text → similar numbers.*
4. Store vectors + original text in a Vector Database

Query Phase (real-time):

1. User asks: "How do I restart the payment service?"
2. Convert query to embedding: $\vec{q} = \text{Embed}(\text{query})$
3. Retrieve top-k most similar chunks (cosine similarity: $\cos(\vec{q}, \vec{v}) = \frac{\vec{q} \cdot \vec{v}}{|\vec{q}||\vec{v}|}$) *This measures the "angle" between two meaning vectors. Closer angle = more relevant.*
4. Inject retrieved chunks + user query into LLM prompt → Get grounded answer

Vector Databases, Choosing the Right One

What is a vector database? A database optimized for storing and *similarity-searching* vector embeddings at scale.

DB	Best For	Notes
Pinecone	Production SaaS	Fully managed, simple API
Weaviate	Hybrid search	Open-source, graph features
pgvector	Existing Postgres	Easiest for teams with Postgres
Chroma	Local dev / prototyping	Easiest to get started
Qdrant	High performance	Rust-based, great filtering

Recommendation for GTC teams: Start with **pgvector** if you already use Postgres. Migrate to Pinecone/Qdrant if scale demands it.

Designing a Knowledge Base, Best Practices

Content Strategy:

- Identify sources: runbooks, Confluence, JIRA history, Slack threads, support tickets, API docs
- Prefer structured, clean text over scanned PDFs or screenshots
- Keep documents versioned and dated, stale context is dangerous

Chunking Strategy:

- Fixed-size: simple, good for homogeneous docs (500 tokens, 10% overlap)
- Semantic: split at paragraph/section boundaries, better for wikis
- Hierarchical: store both summary and detail, allow coarse-then-fine retrieval

Metadata:

- Always store: source URL, date, author, doc type, team
- Use metadata for pre-filtering ("only search runbooks from AppOps team")

Prompting 101

Make everyone effective immediately

How LLMs "Think", Tokens and Context

- LLMs process text as **tokens**, roughly 3/4 of a word on average
- The **context window** is how much text the model can "see" at once, 128K to 1M tokens in modern models
- Everything you want the model to know *must be in the context*: history, instructions, documents
- The model is *stateless* by default, it forgets everything between API calls

Practical implication:

- Put the most important instructions at the *beginning* and *end* of the prompt (recency & primacy effects)
- Don't assume the model knows your codebase, provide relevant snippets
- Token costs money: ≈ $0.003 per 1K input tokens (GPT-4o pricing guide)

Prompt Patterns, Role Prompting

Pattern: Assign the model an expert role to activate relevant knowledge and style.

Basic Prompt

```
Explain Kubernetes pod scheduling.
```

Role Prompt (Better)

```
You are a senior Site Reliability Engineer with 10
years of Kubernetes experience, mentoring a junior
developer.  Explain pod scheduling concisely, using
an analogy.
```

Why it works: The role activates specific patterns from training, an expert's vocabulary, precision, and communication style. **Use in SDLC:** "You are a security code reviewer following OWASP Top 10..."

Prompt Patterns, Few-Shot Prompting

Pattern: Provide 2–3 input/output examples before your actual request.

Few-Shot for Commit Message Generation

`Input: Added null check in user auth handler Output: fix(auth): add null guard for missing user token`[0.4em] `Input: Updated Dockerfile to use slim base image Output: chore(docker): switch to python:3.11-slim base image`[0.4em] `Input: Fixed off-by-one in pagination logic Output:`

Why it works: Examples define the output format, style, and level of detail, far more precisely than any instruction. **Use in SDLC:** Few-shot for test case format, release note style, JIRA ticket format.

Prompt Patterns, Chain-of-Thought

Pattern: Ask the model to *reason step-by-step* before giving the final answer.

Without CoT (bad for reasoning)

```
Is this SQL query safe from injection? [query].
Answer yes or no.
```

With Chain-of-Thought (better)

```
Review this SQL query for injection vulnerabilities.
First, identify all user-controlled inputs. Second,
check if they are parameterized. Third, check for
dynamic string concatenation. Finally, give your
verdict with justification.
```

Why it works: Forces the model to generate intermediate reasoning, reducing errors on complex multi-step tasks. Think of it as asking someone to "show their work."

Prompt Patterns, Structured Output

Pattern: Ask for output in a specific structured format (JSON, YAML, Markdown table).

Structured Output Prompt

```
Analyze this stack trace and return ONLY valid JSON:
{"root_cause": "...", "affected_service": "...",
"suggested_fix": "...", "severity": "P1|P2|P3"}
```

Why it works for engineering:

- Downstream code can parse and act on structured output programmatically
- Reduces hallucinated free-text, enforces schema

Advanced: Use *function calling / tool use* API feature to guarantee valid JSON schema from the model.

Prompting for Engineering Tasks, Examples

- **Debugging:** `''Here is a Python function and the failing test output. Identify the bug and provide a fix with explanation.''[code][output]`
- **Refactoring:** `''Refactor the following function to reduce cyclomatic complexity, improve readability, and add docstrings. Preserve behavior exactly.''[code]`
- **Writing Specs:** `''Given this user story, write a detailed technical specification including: API contract, data model changes, edge cases, and non-functional requirements.''`
- **RCA Analysis:** `''Here are logs from the last 30 minutes before the outage. Use the 5-Whys framework to identify root cause and immediate mitigation.''[logs]`
- **Support Investigation:** `''Customer reports X. Here are the relevant service logs. Identify the user-facing issue, technical cause, and ETA for fix.''[logs]`

AI in the SDLC

Where AI fits across the engineering lifecycle

AI Across the Full SDLC, Overview

Phase	AI Tool	Value
Requirements	LLM + agent	Clarify, refine, generate ACs
Design	LLM + diagram gen	Architecture review, tradeoffs
Coding	Copilot, Cursor	30–40% speed boost
Code Review	PR review agent	Bugs, style, security
Testing	Test gen agent	Unit, E2E, edge cases
Documentation	Doc gen LLM	API docs, READMEs, changelogs
Release	Release notes agent	Summarize PRs into notes
Monitoring	AIOps agents	Anomaly detection, alerting
Incident	RCA agent	Log analysis, 5-Whys
Support	Triage agent	Classify, route, suggest fix

How to Test AI Deliverables

Testing AI is fundamentally different from testing deterministic software.

Key principles:

- **Non-determinism:** Same input → different output each run. Use *range of valid outputs*, not exact match.
- **Evaluation (Evals):** Create a test set of (input, expected behavior) pairs. Measure: Relevance, Faithfulness (grounded in source), Toxicity, Format correctness
- **Human-in-the-loop review:** For high-stakes outputs (code going to prod, customer communications), require human approval
- **Regression suite:** As you improve prompts, run regression to ensure no quality degradation

Tools: Ragas (RAG evaluation), LangSmith (tracing + eval), PromptFoo (prompt regression testing)

MLOps / AI Ops, Operating AI in Production

What is MLOps? The discipline of deploying, monitoring, and maintaining ML/AI systems reliably in production.

- **Model Versioning:** Track which model version is in production; enable rollback
- **Prompt Versioning:** Treat prompts as code, version-controlled, reviewed, tested
- **Observability for LLMs:**
  - Latency per call, token usage, cost per user
  - Quality metrics: hallucination rate, user satisfaction, rejection rate
- **Drift Detection:** Model outputs degrading over time? Monitor quality metrics, re-evaluate regularly.
- **Guardrails:** Input/output validation to block prompt injection, off-topic responses, toxic content
- **Canary Deployments:** Roll out new model/prompt to 5% traffic, validate quality before 100% rollout

Monitoring AI Agents in Production

Agents introduce unique operational challenges:

- **Trace every step:** Log each reasoning step, tool call, and result, not just the final output
- **Tool call auditing:** Which external systems did the agent call? With what parameters? What was returned?
- **Failure modes to monitor:**
  - Infinite loops (agent stuck retrying)
  - Hallucinated tool calls (inventing APIs that don't exist)

  - Cascading failures (agent errors that trigger more agents)

- **Human escalation triggers:** Define conditions under which the agent must pause and request human approval (e.g., destructive database operations, customer-facing communications)
- **Cost monitoring:** Multi-step agents can consume 10×–100× more tokens than single-turn calls

Use Case Workshop

Ground AI in your reality

AI Opportunity Areas for GTC Teams

Use this map to identify your highest-value AI opportunities:

- **Dev:** Code completion, refactoring, doc generation, code review, security scan
- **QA:** Test case generation, test prioritization, bug triage, flaky test analysis
- **AppOps:** Runbook assistant, anomaly detection, alert triage, capacity forecasting
- **Support:** Ticket classification, auto-response drafting, knowledge base Q&A, escalation prediction
- **Data / Ingestion / Monitoring:** Data quality checks, pipeline failure diagnosis, schema inference, anomaly detection

Workshop format: Each group picks one area, drafts: (1) Problem statement (2) Current pain (3) AI-assisted future state (4) Success metric

Use Case Template

Fill this in for your area:

- **Problem Statement:** (1 sentence, what is broken today?)
- **Current Pain:** (quantify if possible, hours/week, error rate, MTTR)
- **AI-Assisted Future State:** (what does the workflow look like with AI?)
- **AI Approach:** Chatbot? RAG? Agent? Fine-tuned model?
- **Data Needed:** What documents, logs, tickets, or code does the AI need access to?
- **Success Metric:** How will you measure if it worked? (e.g., 40% reduction in triage time)
- **Risk/Constraint:** Data privacy concern? Need human in loop? Hallucination risk?

Build a Simple Internal AI App

Remove fear. Build confidence.

Three App Examples to Build

Option A: Internal Doc Q&A Bot (RAG)

- Upload your team's wiki / runbooks
- Ask natural language questions, get grounded answers with citations
- Stack: Python + LangChain + ChromaDB + Streamlit + OpenAI API

Option B: Runbook Assistant Agent

- Given an alert name, the agent finds the relevant runbook and executes steps
- Stack: LangChain Agent + custom tools (Confluence API, kubectl, PagerDuty)

Option C: Log / Ticket Analysis Bot

- Paste a log snippet or support ticket; get structured diagnosis + suggested fix
- Stack: OpenAI API + structured output + React UI

Sample Code, RAG Bot in 20 Lines

```
from langchain.document_loaders import
     DirectoryLoader
from langchain.text_splitter import
     RecursiveCharacterTextSplitter
from langchain.embeddings import OpenAIEmbeddings
from langchain.vectorstores import Chroma
from langchain.chains import RetrievalQA
from langchain.llms import OpenAI

# 1. Load your internal docs
docs = DirectoryLoader("./runbooks/").load()

# 2. Chunk and embed
chunks = RecursiveCharacterTextSplitter(
    chunk_size=500).split_documents(docs)
db = Chroma.from_documents(chunks,
     OpenAIEmbeddings())

# 3. Create RAG chain
qa = RetrievalQA.from_chain_type(
    llm=OpenAI(), retriever=db.as_retriever())

# 4. Answer questions
print(qa.run("How do I restart the auth service?"))
```

(3-Week) Team Assignment

Build a practical AI solution for one SDLC phase

Team Assignment, Structure and Deliverables

Objective: Each group builds a practical AI solution that improves one SDLC phase.

Example projects:

- PR review agent — Test case generation agent — Support ticket triage agent
- RCA agent — Requirements → design → code agent — Data quality monitoring agent

Deliverables per team:

- **Working demo** (live or recorded)
- **Repo with:** code, prompt design (versioned), architecture diagram, cost estimate
- **10-min presentation:** Business impact, practicality, reusability

Format: Team picks use case + leader. Optional async weekly check-in. Lightweight, respect leadership schedules.

AI ROI Model & FinOps

Evaluating AI investments, critical for executives

Cost Components of an AI System

Every AI system has these cost buckets, understand them before committing:

- **Model / API Cost:** Pay-per-token (OpenAI, Anthropic, Google) or self-hosted GPU cost
  - GPT-4o: ≈ \$2.50 / 1M input tokens, \$10 / 1M output tokens
  - Llama 3 self-hosted: \$0 tokens + GPU cost (≈ \$1–3/hr on A100)
- **Compute:** GPU instances for inference (if self-hosted), vector DB, orchestration services
- **Storage:** Vector DB storage, prompt/log archives
- **Security:** Data encryption, VPC, audit logging, DLP tools
- **Tooling:** LangSmith, Langfuse, Promptfoo, dev tooling licenses
- **Human Cost:** Prompt engineers, ML engineers, data labelers for evals, governance team

Build vs. Buy vs. Integrate, Decision Framework

Option	When	Cost	Risk
Use SaaS AI tool	Commodity task	Low	Lock-in
Integrate LLM API + RAG	Custom on your data	Medium	Dependency on provider
Fine-tune existing LLM	High-volume, specific task	High	Needs labeled data
Train own LLM	Core IP, extreme scale	Very High	>$1M budget

Recommendation for most GTC use cases: Integrate LLM API + RAG.

- 90% of internal use cases solved with API + RAG
- Fine-tune only when you have >10K labeled examples and quality still insufficient
- Train your own model: almost never justified unless you are a foundation model company

AI ROI, How to Think About It

ROI formula (simplified):

$$\text{ROI} = \frac{\text{Value Gained} - \text{Total AI Cost}}{\text{Total AI Cost}} \times 100\%$$

This means: for every rupee spent on AI, how many rupees of value do you get back?

Value can be:

- **Time saved:** 40% reduction in PR review time for 20 engineers = 20 × 0.4 × hourly rate per week
- **Quality gains:** Bugs caught by AI before production = avoided incidents (quantify per P1 cost)
- **Revenue impact:** Faster feature delivery = earlier market capture
- **Cost avoidance:** AI handles Tier-1 support = fewer support hires

Productivity metric frameworks: SPACE (Satisfaction, Performance, Activity, Communication, Efficiency), DORA metrics with AI

FinOps for AI, Cost Control Strategies

Token cost is your primary lever, optimize it:

- **Use the right model for the task:** Route simple queries to GPT-4o-mini ($0.15/1M) not GPT-4o ($2.50/1M), 16× cheaper
- **Prompt caching:** Cache repeated system prompts; Anthropic offers 90% discount on cached tokens
- **Limit output tokens:** Set `max_tokens`, output is 3–4× more expensive than input
- **Batching:** Use Batch API for non-real-time tasks, 50% cost reduction
- **Monitoring dashboards:** Track cost per user, per feature, per team, set budgets and alerts
- **Usage governance:** Define who can use which model tier; require justification for expensive models

Target: Set cost-per-interaction KPI before launching any AI feature to production.

Pricing AI Products, Own LLM vs. Integrate

If you are *building an AI product* to sell:

- **Integrated LLM model:** Pass-through API cost + margin. Predictable per-call pricing. Fast to market.
  - *Risk:* Provider price changes affect your margins. Dependency on third party uptime.
- **Own / self-hosted LLM:** Fixed GPU infrastructure cost, variable at scale, better unit economics after breakeven
  - *Risk:* Heavy upfront infra investment, MLOps team needed, model maintenance burden

Pricing models for AI products:

- Per-query pricing (e.g., $X per 1000 API calls)
- Seat-based (per user/month), bundle AI cost into subscription
- Usage-based with tiers, free tier limited, paid tier unlimited (freemium)
- Outcome-based, pay per resolved ticket, per detected bug (hardest to implement)

AI Governance, Risk & Ethics

Enterprise-grade view

AI Governance, What Must Be in Place

An AI governance framework answers:

- **AI Usage Policy:** What is allowed? What requires review? What is prohibited?
- **Approval Workflow:** How does a team get approval to deploy an AI feature to production?
- **Review & Audit:** How are AI outputs reviewed? How long are interactions logged? Who can audit?
- **Incident Response:** What happens when AI causes a production issue or generates harmful content?
- **Vendor Management:** Which AI providers are approved? What are the contractual data-use clauses?
- **Training & Certification:** Are engineers trained on AI policy before building AI features?

Key AI Risks for Engineering Leaders

- **Hallucinations:** Model confidently states wrong information. *Mitigation: RAG grounding, human review for high-stakes outputs, show sources*
- **Data Leakage:** Sensitive data sent to external LLM APIs. *Mitigation: DLP, approved-only APIs, no PII in prompts policy*
- **Bias & Unfairness:** AI trained on biased data produces biased outputs. *Examples: Amazon hiring tool (2018), facial recognition errors for minorities*
- **Auditability:** Cannot explain why AI made a decision, regulatory risk. *Mitigation: Log all inputs/outputs, use explainable models where regulation demands*
- **Regulatory Exposure:**
  - EU AI Act (2024): Risk-based regulation; high-risk AI systems require conformity assessment
  - GDPR: AI decisions using personal data require legal basis + right to explanation

AI Bias, Fairness & Environmental Cost

Bias Examples:

- Amazon hiring AI (2018): downgraded female candidates, trained on male-dominated historical data
- Facial recognition: higher error rates for people with darker skin tones
- Loan approval AI: replicating historical redlining patterns

What's being done: Fairness metrics, diverse training data, EU AI Act mandatory bias audits.

Environmental Cost:

- Training GPT-3: ≈ 552 tonnes CO_2 equivalent
- Running large LLMs: significant water cooling consumption in data centers
- Counter-balance: AI optimizing energy grids (DeepMind: 15% energy reduction), climate modeling

Message: "Innovation with responsibility."

Operating Model & Adoption Strategy

Often missed, but crucial

How to Drive AI Adoption, The Leadership Playbook

What kills AI adoption:

- Random experimentation, 20 teams, 20 tools, no shared learning
- Fear and distrust, engineers worried AI will replace them
- No clear ownership, everyone waits, no one acts

What works:

- **AI Champions:** Identify 1–2 enthusiastic engineers per team. Give them time, budget, and air cover to experiment and teach.
- **Internal AI Platform:** Centralized, pre-approved AI tools, shared prompt library, shared embedding pipeline. Reduces redundant work, ensures compliance.
- **Show wins fast:** Pick one high-value, low-risk use case per team. Show a working demo within 3 weeks. Builds confidence.
- **Normalize AI in reviews:** Include AI-assisted PRs in code reviews. Normalize AI-generated first-draft docs.

What Leadership Must Change

Estimation and planning:

- Sprint velocity estimates need to account for AI-assisted development (adjust story points)
- Some tasks that took 2 days now take 2 hours, your roadmap timelines change

Review processes:

- AI-generated code still needs review, but the focus shifts from typos to logic and architecture
- Add AI output review as an explicit step in Definition of Done

Quality gates:

- Require eval scores above threshold before AI feature goes to production
- Mandatory human-in-loop for customer-facing AI outputs in first 3 months

Expectations:

- AI is a *force multiplier*, not a replacement. Outcomes improve; team size stays (or grows for higher-value work).

The Future of AI in Engineering

Where AI is going, and what to prepare for

Physical AI, The Next Frontier

AI is leaving the screen and entering the physical world:

- **Robotics:** Boston Dynamics + AI = robots that navigate unstructured environments. NVIDIA Isaac platform for robot training.
- **Autonomous Vehicles:** Waymo, Tesla FSD, AI driving at scale
- **Smart Manufacturing:** Computer vision for defect detection; robotic arms with LLM-based task planning
- **Healthcare:** Robotic surgery (da Vinci AI), AI-guided drug discovery (AlphaFold, Nobel Prize 2024)
- **NVIDIA's focus (2025):** "Physical AI" as the next major platform shift, combining foundation models with robotics

For engineering leaders: Physical AI opens new product opportunities. It also introduces new safety-critical requirements, model correctness is now literally life-or-death.

AGI and What Lies Ahead

What is AGI? Artificial General Intelligence, a system that can perform *any intellectual task* a human can, at human-level or better.

Timeline predictions:

- **Optimists (2030s):** Sam Altman, Demis Hassabis, we are on a rapid exponential curve
- **Skeptics (decades or never):** Yann LeCun, LLMs lack grounding, causality, world models

What we do know:

- Current AI is *narrow but deep*, superhuman at specific tasks, poor at reasoning across domains
- Key unsolved challenges: common-sense reasoning, causal understanding, reliable long-horizon planning

Leader takeaway: Plan for *AI-augmented engineering* in the near term, not AGI. Invest in AI literacy, tooling, and governance now.

Jobs of the Future, Engineering Edition

Jobs AI creates:

- Prompt Engineers, AI Application Developers, MLOps / AI Ops Engineers
- AI Safety Engineers, AI Ethics Officers, Data Curators, Eval Specialists

Jobs transforming:

- Software Engineer → AI-augmented engineer (reviewer, director, architect)
- QA Engineer → AI test strategist (designs eval frameworks, not individual test cases)
- SRE → AI Ops engineer (manages AI pipelines alongside infra)
- Tech Lead → AI adoption champion and AI architecture decision maker

Skills to develop now: Prompt engineering, RAG system design, AI eval design, AI governance, cost management for LLMs.

AI Limits, Know What It Cannot Do

- **No true abstract reasoning:** Can simulate reasoning but lacks genuine causal understanding
- **Accuracy limits:** Every model has a hallucination rate, never trust without verification for critical tasks
- **Hard to capture tacit knowledge:** Things your team "just knows" from years of experience are not in training data
- **Interpretability:** Large models are black boxes, explaining *why* a decision was made is still hard
- **Context length limits:** Even 1M token models cannot truly "understand" a million-token codebase, chunking introduces blind spots
- **Real-time awareness:** Without RAG or tools, models are frozen at training cutoff

Wrap-Up, AI Roadmap for Your Org

When to Use What, Decision Tree

- **Clear rules, deterministic logic?** → Traditional programming
- **Pattern recognition in structured data?** → Classical ML (XGBoost, sklearn)
- **Images, speech, complex unstructured data?** → Deep Learning / CNN / RNN
- **Text generation, Q&A, summarization, code?** → LLM via API
- **Q&A on your private documents?** → RAG + LLM
- **Multi-step autonomous task on internal systems?** → LLM Agent + MCP
- **Volume: 10M+ calls/day, very specific task?** → Fine-tune SLLM
- **Real-time physical-world control?** → Physical AI / Robotics

Your 6–12 Month AI Roadmap Template

Agree on these with your leadership team:

- **3–5 Priority Use Cases:** Rank by ROI × feasibility. Start with quick wins.
- **Investment Areas:**
  - Tooling (licenses, API access, vector DB)
  - People (1–2 AI champions per team, 1 MLOps engineer)
  - Training (this program, prompt engineering workshops)
- **Governance Model:** Finalize AI usage policy, approval workflow, audit mechanism
- **Milestones:**
  - Month 1–2: Internal AI policy published. AI champions identified. First RAG bot live.
  - Month 3–4: 2–3 team-level AI features in production with evals.
  - Month 5–6: Internal AI platform (shared tools, shared prompt library). FinOps dashboards live.
  - Month 7–12: AI in every SDLC phase. Governance mature. ROI measured.

Getting Started, Your Action Plan

For Tech Leads & Architects:

- Build: a RAG bot on your team's runbooks this sprint
- Evaluate: GitHub Copilot / Cursor for your team (1-month trial)
- Read: LangChain docs, OpenAI Cookbook

For Engineering Managers:

- Identify: your team's AI champion
- Draft: team-level AI usage policy (use this deck as input)
- Measure: baseline DORA metrics before AI adoption to quantify ROI after

For All Leaders:

- Hands-on: Spend 30 min/day with Claude or ChatGPT for real work tasks
- Stay current: Subscribe to *The Batch* (DeepLearning.AI), *TLDR AI*
- Think critically: Question every AI claim, ask for the eval results

Resources & Further Learning

Courses: fast.ai (free), DeepLearning.AI Short Courses, Coursera ML Specialization (Andrew Ng)

Books: *Hands-On ML* (Géron), *Building LLM Applications* (Iusztin), *AI Engineering* (Chip Huyen, 2025)

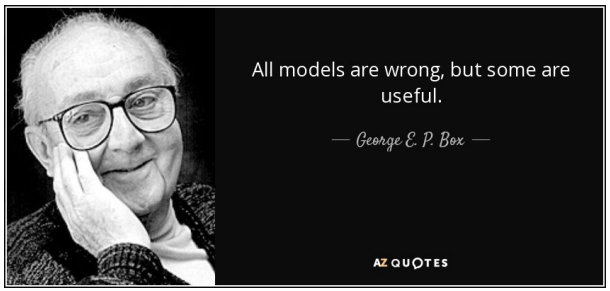
Hands-on: OpenAI Cookbook (github), LangChain docs, Hugging Face tutorials

Benchmarks & Research: arXiv.org, Papers with Code, LMSYS Chatbot Arena Leaderboard

Newsletters: The Batch, TLDR AI, Import AI (Jack Clark), Ahead of AI (Sebastian Raschka)

Communities: Hugging Face Discord, r/MachineLearning, local AI meetups

Tools to try today: Claude.ai, ChatGPT, GitHub Copilot (free tier), Google Colab, Cursor IDE



Copyleft ⓒ Send suggestions to yogeshkulkarni@yahoo.com